Fiche de Synthèse – Projet MLOps : Du Notebook à la Production

1. Le trajet du modèle : Du Notebook à Internet

Le cycle de vie du modèle s'articule autour d'une chaîne MLOps automatisée :

- **Développement & Expérimentation :** Le modèle d'intelligence artificielle est d'abord entraîné et évalué dans un Jupyter Notebook. Les artefacts générés (fichiers modèle) sont sauvegardés dans le dossier `model/`.
- **API FastAPI :** Une interface REST est développée avec FastAPI (`app.py`) pour exposer les endpoints nécessaires (`/health` pour la vérification de l'état du service et `/predict` pour l'inférence).
- **Conteneurisation Docker :** L'application, ses dépendances (`requirements.txt`) et les modèles sont empaquetés dans une image Docker légère et isolée.
- **Pipeline CI/CD (GitHub Actions) :** À chaque `git push`, le workflow exécute automatiquement les tests unitaires, contrôle le seuil de qualité, construit l'image Docker, puis la pousse sur le registre privé **Azure Container Registry (ACR)** (`hamidRegistre`).
- **Déploiement Cloud :** L'image conteneurisée est déployée sur **Azure Container Apps** (`movie-container`), rendant l'API accessible publiquement sur Internet.

2. La barrière de qualité : Pourquoi ce seuil ?

Afin d'éviter tout déploiement d'un modèle sous-performant en production, un script de contrôle de qualité a été intégré au pipeline CI/CD :

- **Objectif :** Garantir que seules les versions répondant aux exigences métiers minimale (ex. exactitude ou F1-score au-dessus d'un seuil défini) soient conteneurisées.
- **Validation :** Lors du test de la *« chaîne rouge »* avec un seuil volontairement rendu impossible à atteindre, le pipeline a immédiatement bloqué le déploiement, prouvant l'efficacité de cette barrière de sécurité.

3. Déploiement manuel vs Automatique

- **Pourquoi déployer/contrôler à la main ?** Dans ce projet, le redéploiement et la gestion des révisions (rollback) ont été effectués manuellement depuis le portail Azure afin de conserver une maîtrise totale sur les versions d'images déployées et de valider pas à pas le comportement de l'infrastructure lors d'un retour arrière.
- **Pour un déploiement 100 % automatique (Continuous Deployment) :** Il faudrait ajouter une étape de déploiement automatique dans le fichier GitHub Actions (à l'aide des CLI Azure) et configurer un mécanisme de passage de trafic progressif (déploiement *Canary* ou *Blue-Green*) avec des webhooks d'alerte en cas de régression.

4. Limites du travail

Bien que le pipeline soit fonctionnel et sécurisé, plusieurs axes d'amélioration subsistent :

- **Monitoring en production :** Absence de suivi en temps réel des dérives de données (*data drift*) et des performances du modèle au fil du temps.
- **Gestion avancée des secrets :** Utilisation de secrets GitHub simples au lieu d'un coffre-fort dédié comme *Azure Key Vault*.
- **Gestion des coûts & quotas :** Restrictions observées sur l'analyse fine des coûts en raison du type d'abonnement cloud utilisé.